



N reinas

Nombre de los alumnos:

SEBASTIAN ALCANTAR PEREZ

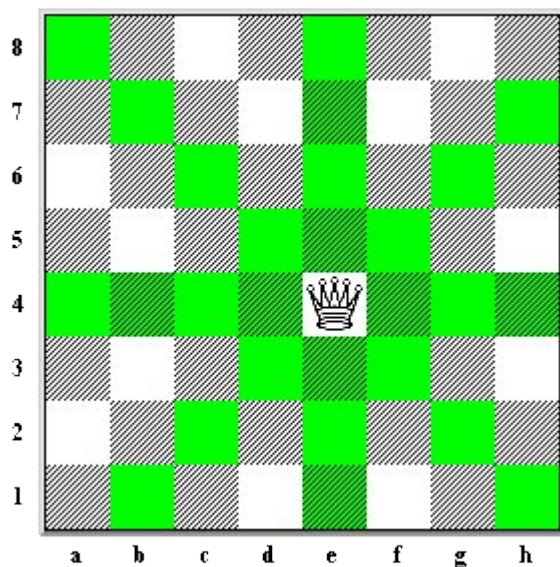
ABEL ALEJANDRO MARTINEZ LIZARAGA

Nombre del maestro: ZURIEL DATHAN MORA FELIX

DESCRIPCION DEL PROBLEMA

Para este problema tienes que pensar en un tablero de ajedrez, una vez hecho esto dado un número N , intenta poner N reinas en un tablero $N \times N$.

En caso de que no tenga el conocimiento, una reina en el ajedrez se puede mover de izquierda a derecha, arriba y abajo y a los diagonales. Para que quede más claro, en la siguiente imagen se ve de manera gráfica el movimiento de una reina en un tablero de ajedrez:



Entonces, el problema hace que las reinas colocadas no se choquen entre sí, en este caso, nos piden resolver el problema para un valor de N de ocho, por lo que tendríamos 8 reinas en un tablero de 8×8 .

REPRESENTACION DE P Y S

P (Espacio del problema o estado): Para optimizar la búsqueda, el problema se representa como un arreglo unidimensional, donde el índice representa la columna y el valor representa la fila en la que se encuentra la reina. Para evitar conflictos de filas y columnas desde el inicio, el estado inicial siempre será una permutación de los números del 0 al 7 (por ejemplo, [0, 4, 7, 5, 2, 6, 1, 3]). En otras palabras, haremos que no se puedan repetir los números en el arreglo, para evitar que colacionen las columnas y las filas. Con esta representación, solo necesitamos preocuparnos por resolver las colisiones diagonales.

S (Solución): Una solución S es aquel estado del espacio P donde la función objetivo (la cantidad de pares de reinas que se atacan mutuamente en las diagonales) es igual a 0.

PROPUESTA DEL ALGORITMO EN PSEUDOCODIGO

Función `BusquedaTabu(max_iteraciones, tamaño_tabu)`:

```
solucion_actual = generar_solucion_inicial_aleatorio()
```

```
mejor_solucion = solucion_actual
```

```
lista_tabu = []
```

For iteracion desde 1 hasta max_iteraciones:

IF calcular_colisiones(solucion_actual) == 0:

Return estado_actual (Solución óptima encontrada)

vecinos = generar_vecindario(solucion_actual)

mejor_vecino = Nulo

menor_colicion_vecino = Infinito

mejor_movimiento = Nulo

Para cada (vecino, movimiento) en vecinos:

colisiones = calcular_colisiones(vecino)

IF (movimiento NO está en lista_tabu) O (colisiones < calcular_colisiones(mejor_solucion)):

IF colisiones < menor_colisiones_vecino:

menor_colicion_vecino = colisiones

mejor_vecino = vecino

mejor_movimiento = movimiento

solucion_actual = mejor_vecino

IF menor_colicion_vecino < calcular_colisiones(mejor_solucion):

mejor_solucion = mejor_vecino

Agregar mejor_movimiento a lista_tabu

IF tamaño(lista_tabu) > tamaño_tabu:

Eliminar elemento más antiguo de lista_tabu

Retornar mejor_solucion

IMPLEMENTACION CON PYTHON

El código implementado y documentado en python está en este repositorio de github:
<https://github.com/SebastianAlcantar/TOPICOS-DE-IA>